

Implementation of Half Adder with testbench using Verilator (All Modelling styles)

Introduction

This lab provides an introduction to designing a simple half adder using Verilator. You will learn to create the design, write a corresponding testbench, simulate the behavior, and analyze the output using waveform tools.

Objectives

After completing this lab, you will be able to:

- Write Verilog code for a simple half adder in Gate level , Dataflow, and Behavioral, modelling in Verilog.
- Develop a testbench to verify the design.
- Simulate the design using Verilator.
- Observe and analyze the half adder behavior using waveform output.

Procedure:

This lab consists of the following steps:

1. Creating the lab directory
2. Creating new project directory
3. Writing the design file
4. Writing testbench file
5. Simulating the design using Verilator
6. Analyzing the waveform dump file

Block Diagram of Half Adder:

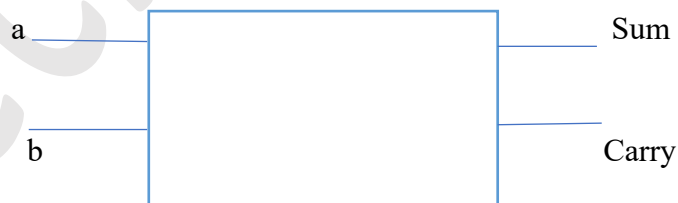


Fig: Block Diagram of Half Adder

Equation: **Sum = A XOR B**

Carry = A AND B

Gate-Level Modeling

Step 1: Navigate back to the Lab Directory

1.1 Navigate to the lab directory

```
cd ~/verilator_lab_project
```

1.2 Create a new project directory for half adder in gate-level modeling

```
mkdir half_adder_gatelevel
```

1.3 Navigate into the project directory

```
cd half_adder_gatelevel
```

Step 2: Create the Design File

2.1 Open GVim to write the half adder design in gate-level modeling

```
gvim half_adder_gatelevel.v
```

```
// Half Adder - Gate Level Modeling
// This model uses primitive logic gates

`timescale 1ns/1ps

module half_adder_gatelevel(
    input a,          // First input bit
    input b,          // Second input bit
    output sum,        // Sum output
    output carry       // Carry output
);

// Primitive gate instantiation
xor u1(sum, a, b);    // sum = a XOR b
and u2(carry, a, b);  // carry = a AND b

endmodule
```

2.3 Save and exit the editor

```
:wq!
```

Step 3: Create the Testbench File

3.1 Open GVim

```
gvim half_adder_gatelevel_tb.v
```

3.2 Testbench Code

```
// Testbench for Half Adder - Gate Level Modeling

`timescale 1ns/1ps

module half_adder_gatelevel_tb;

reg a, b;
wire sum, carry;

// Instantiate Unit Under Test (UUT)
half_adder_gatelevel uut (
    .a(a),
    .b(b),
    .sum(sum),
    .carry(carry)
);

initial begin

$display("Gate Level Half Adder Test");
$display("A B | Sum Carry");
$display("-----");

a = 0; b = 0; #10 $display("%b %b | %b %b", a, b, sum,
carry);
a = 0; b = 1; #10 $display("%b %b | %b %b", a, b, sum,
carry);
a = 1; b = 0; #10 $display("%b %b | %b %b", a, b, sum,
carry);
a = 1; b = 1; #10 $display("%b %b | %b %b", a, b, sum,
carry);

$finish;

end

initial begin
$dumppfile("half_adder_gatelevel.vcd");
$dumpvars(0, half_adder_gatelevel_tb);
end

endmodule
```

3.3 Save and exit

:wq!

3.4 Verify files : ls

Step 4: Simulate the Design Using Verilator

4.1 Compile

```
verilator --binary -j 0 -Wall half_adder_gatelevel.v \
half_adder_gatelevel_tb.v --top half_adder_gatelevel_tb \
--timing --CFLAGS "-std=c++20" --trace
```

4.2 If errors occur, fix them and rerun.

4.3 Check generated directory obj_dir : ls

4.4 Navigate into object directory

```
cd obj_dir
```

4.5 Build simulation binary

```
make -f Vhalf_adder_gatelevel_tb.mk Vhalf_adder_gatelevel_tb
```

4.6 Run simulation

```
./Vhalf_adder_gatelevel_tb
```

4.7 Observe terminal output.

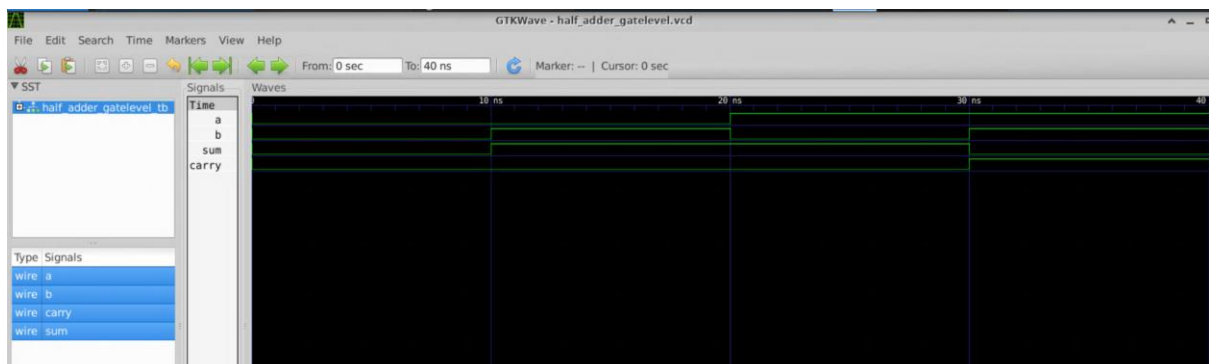
```
Gate Level Half Adder Test
a b | sum carry
-----
0 0 | 0 0
0 1 | 1 0
1 0 | 1 0
1 1 | 0 1
- half adder gatelevel tb.v:31: Verilog $finish
```

Step 5: View Waveform Output

5.1 Open GTKWave

```
gtkwave half_adder_gatelevel.vcd
```

5.2 Observe how sum and carry change with inputs a and b.



2) Dataflow Modelling

Steps:

Step 1: Create a Lab Directory

1.1: Create a new directory inside your lab workspace:

```
mkdir verilator_lab_project
```

1.2: Navigate to the lab directory:

```
cd verilator_lab_project
```

1.3: Create a new project directory for half adder in dataflow style of modelling

```
mkdir half_adder_dataflow
```

1.4: Navigate into the project directory for half-adder in dataflow style of modelling

```
cd half_adder_dataflow
```

Step 2: Create the Design File

2.1: open GVim to write the half adder design in dataflow style of modelling

```
gvim half_adder_dataflow.v
```

Code of Half Adder:

// Half Adder - Dataflow Modelling

```
// This model uses continuous assignment with assign statements
```

```
`timescale 1ns/1ps // Time unit = 1ns, time precision = 1ps
```

```
module half_adder_dataflow(
```

```
    input a,                // First input bit
```

```
    input b,                // Second input bit
```

```
    output sum,             // Sum output
```

```
    output carry            // Carry output
```

```
);
```

```
    assign sum = a ^ b;     // XOR operation for sum
```

```
    assign carry = a & b;   // AND operation for carry
```

```
endmodule
```

2.3: Save and exit the editor

Type :wq! and press Enter

Step 3: Create the testbench File

3.1: Open GVim to create the testbench

```
gvim half_adder_dataflow_tb.v
```

3.2: Write the testbench code to test the half adder behavior

// Testbench for Half Adder - Dataflow Modeling

```
`timescale 1ns/1ps // Time unit = 1ns, time precision = 1ps
module half_adder_dataflow_tb;
    reg a, b; // Input test signals
    wire sum, carry; // Outputs from the half adder
    // Instantiate Unit Under Test (UUT)
    half_adder_dataflow uut (
        .a(a),
        .b(b),
        .sum(sum),
        .carry(carry)
    );
    // Test stimulus block
    initial begin
        $display("Dataflow Half Adder Test");
        $display("A B | Sum Carry");
        $display("-----");
        a = 0; b = 0; #10 $display("%b %b | %b %b", a, b, sum,
carry); // 0 + 0
        a = 0; b = 1; #10 $display("%b %b | %b %b", a, b, sum,
carry); // 0 + 1
        a = 1; b = 0; #10
        $display("%b %b | %b %b", a, b, sum, carry); // 1 + 0
        a = 1; b = 1; #10 $display("%b %b | %b %b", a, b, sum,
carry); // 1 + 1
```

```
$finish; // End simulation

end

// Dump waveform data for GTKWave

initial begin

    $dumpfile("half_adder_dataflow.vcd"); // Output VCD file

    $dumpvars(0, half_adder_dataflow_tb); // (Dump all
variables in testbench)

end

endmodule
```

3.3: Save and exit the editor

Type : wq! and press Enter

3.4: List the files to confirm they are saved

ls to check the files.

Step 4: Simulate the Design Using Verilator

4.1: Run Verilator to compile the design and testbench

```
verilator --binary -j 0 -Wall half_adder_dataflow.v
half_adder_dataflow_tb.v --top half_adder_dataflow_tb --timing
--CFLAGS "-std=c++20" --trace
```

4.2: If any errors occur, resolve them and rerun the command.

4.3: After successful compilation, a new directory obj_dir will be created

ls

4.4: Navigate into the obj_dir

cd obj_dir

4.5: Build the simulation binary

```
make -f Vhalf_adder_dataflow_tb.mk Vhalf_adder_dataflow_tb
```

4.6: After compilation, an executable named Vhalf_adder_dataflow_tb will be generated

4.7: Run the simulation

```
./Vhalf_adder_dataflow_tb
```

4.8: Observe the simulation output in the terminal

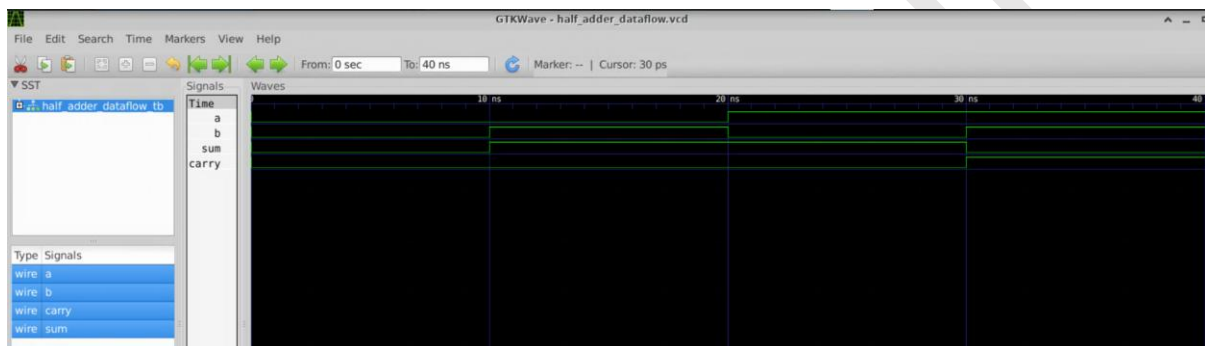
```
Dataflow Half Adder Test
A B | Sum Carry
-----
0 0 | 0 0
0 1 | 1 0
1 0 | 1 0
1 1 | 0 1
- half_adder_dataflow_tb.v:24: Verilog $finish
```

Step 5: View Waveform Output

5.1: Launch GTKWave to view the waveform file

```
gtkwave half_adder_dataflow.vcd
```

5.2: Examine how the half adder behaves with change in the inputs a and b.



3) Behavioral Modelling

Steps:

Step 1: Navigate back to the Lab Directory

1.1: Navigate to the lab directory:

```
cd ~/verilator_lab_project
```

1.2: Create a new project directory for half adder in behavioral style of modelling

```
mkdir half_adder_behavioral
```

1.3: Navigate into the project directory for half-adder in behavioral style of modelling

```
cd half_adder_behavioral
```

Step 2: Create the Design File

2.1: Open GVim to write the half adder design in behavioral style of modelling

```
gvim half_adder_behavioral.v
```

Code:

// Half Adder - Behavioral Modeling

// This model uses an always block and procedural assignment

```
`timescale 1ns/1ps // Time unit = 1ns, time precision = 1ps

module half_adder_behavioral(
    input a,          // First input bit
    input b,          // Second input bit
    output reg sum,    // Sum output (a XOR b)
    output reg carry   // Carry output (a AND b)
);
    //Combinational logic block triggered on changes to 'a' or 'b'
    always @(a or b) begin
        sum = a ^ b;    // XOR for sum
        carry = a & b;   // AND for carry
    end
endmodule
```

2.3: Save and exit the editor

Type :wq! and press Enter

Step 3: Create the testbench File

3.1: Open GVim to create the testbench

```
gvim half_adder_behavioral_tb.v
```

3.2: Write the testbench code to test the half adder behavior

Code:

// Testbench for Half Adder - Behavioral Modeling

```
`timescale 1ns/1ps // Time unit = 1ns, time precision = 1ps

module half_adder_behavioral_tb;
    // Declare test inputs as reg and outputs as wire
    reg a, b;
    wire sum, carry;
```

```
// Instantiate the Unit Under Test (UUT)

half_adder_behavioral uut (

    .a(a),

    .b(b),

    .sum(sum),

    .carry(carry)

);

// Stimulus block

initial begin

    $display("Behavioral Half Adder Test");

    $display("A B | Sum Carry");

    $display("-----");

    // Apply all input combinations

    a = 0; b = 0; #10 $display("%b %b | %b %b", a, b, sum,
carry); // 0 + 0

    a = 0; b = 1; #10 $display("%b %b | %b %b", a, b, sum,
carry); // 0 + 1

    a = 1; b = 0; #10 $display("%b %b | %b %b", a, b, sum,
carry); // 1 + 0

    a = 1; b = 1; #10 $display("%b %b | %b %b", a, b, sum,
carry); // 1 + 1

    $finish; // Ends the simulation

end

// Dump VCD waveform for GTKWave

initial begin

    $dumpfile("half_adder_behavioral.vcd");// (VCD output
filename)

    $dumpvars(0, half_adder_behavioral_tb); // (Dump all
signals in this testbench)

end

endmodule
```

3.3: Save and exit the editor

Type :wq! and press Enter

3.4: List the files to confirm they are saved

```
ls
```

Step 4: Simulate the Design Using Verilator

4.1: Run Verilator to compile the design and testbench

```
verilator --binary -j 0 -Wall half_adder_behavioral.v  
half_adder_behavioral_tb.v --top half_adder_behavioral_tb  
--timing --CFLAGS "-std=c++20" --trace
```

4.2: If any errors occur, resolve them and rerun the command.

4.3: After successful compilation, a new directory obj_dir will be created

```
ls
```

4.4: Navigate into the obj_dir

```
cd obj_dir
```

4.5: Build the simulation binary

```
make -f Vhalf_adder_behavioral_tb.mk Vhalf_adder_behavioral_tb
```

4.6: After compilation, an executable named Vhalf_adder_behavioral_tb will be generated

4.7: Run the simulation

```
./Vhalf_adder_behavioral_tb
```

4.8: Observe the simulation output in the terminal

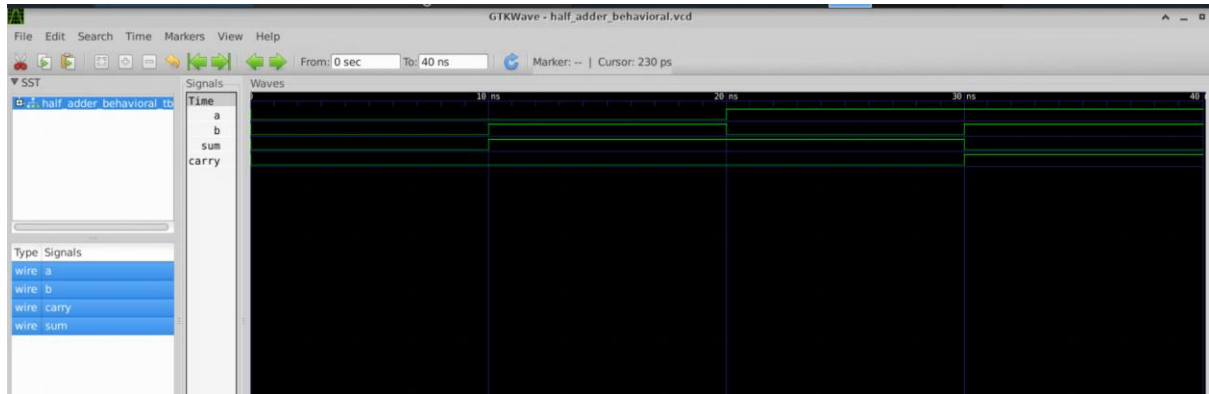
```
Behavioral Half Adder Test  
A B | Sum Carry  
-----  
0 0 | 0 0  
0 1 | 1 0  
1 0 | 1 0  
1 1 | 0 1  
- half_adder_behavioral_tb.v:26: Verilog $finish
```

Step 5: View Waveform Output

5.1: Launch GTKWave to view the waveform file

```
gtkwave half_adder_behavioral.vcd
```

5.2: Examine how the half adder behaves with change in the inputs a and b.



Conclusion

In this lab, you implemented and simulated a simple half-adder using verilator in all the Verilog modeling styles. By analyzing the simulation and waveform output, you observed the behavior of half adder. This lab enhanced your understanding of basic combinational logic design and open-source digital simulation tools.